

Tangent-Space Methods for Nonlinear Control and Biomechanics

Volume V: Practical Application

The Companion Implementation Platform

Preface

Draft Notice. This volume is under active development. Chapter 10 contains a tested three-link prescribed-motion example with explicitly scoped extensions. Other chapters retain preliminary material; the volume does not yet provide a fully implemented or experimentally validated end-to-end golf pipeline.

This is a book about *doing*. Volumes 0–IV developed the theory of motion: from differential geometry through nonlinear control, biomechanics, and motor control. This volume explores how those tools can be connected in computational investigations.

The companion project, UpstreamDrift, is the setting for the software integration discussions in this volume. Engine interfaces and executable examples must be checked against the selected software revision. The capstone here runs the repository’s local three-link model and does not require every engine discussed elsewhere in the volume.

Disclosure. UpstreamDrift is developed by the authors of this textbook. It is not a production simulation engine; it is an educational platform designed to accompany this text and make the theory from Volumes 0–IV executable. The underlying physics computations are delegated to the established engines listed above.

The capstone distinguishes executable model calculations from proposed optimization, muscle, robustness and variability extensions. Its data are synthetic. Each extension needs its own implementation, assumptions and validation before it can support a scientific conclusion.

Contents

Preface	i
Nomenclature	iv
1 The Companion Platform	1
1.1 Architecture and Capability Boundaries	1
1.1.1 What the Engine Names Mean	1
1.2 State, Force and Time Contracts	2
1.3 A Fully Specified Pendulum	2
1.4 Connecting the Example to the Swing	4
2 Physics Engine Comparison	5
2.1 Establish Model Equivalence First	5
2.2 Measure the Right Error	5
2.3 Contact Law, Solver and Integrator Are Separate Choices	6
2.4 A Reproducible Comparison Record	7
3 Building a Multibody Model	9
3.1 Formats and Their Boundaries	9
3.2 Frames, Inertia and Physical Consistency	9
3.3 A Complete Cylinder Pendulum	10
3.4 Extending to a Seven-Segment Golf Model	11
4 Forward and Inverse Simulation	12
4.1 Forward Dynamics With the Companion Platform	12
4.2 Simulation Loop and Integration Schemes	14
4.3 Inverse Dynamics Pipeline	14
5 Trajectory Optimization in Practice	15
5.1 DDP/iLQR Implementation	15
5.2 Application: Optimal Pendulum Swing-Up	17
5.3 Application: Optimal Pendulum Swing-Up	18
5.4 Application: Golf Swing Optimization	18
6 Controller Design and Testing	19
6.1 PD Control With Gravity Compensation	19
6.2 Trajectory Tracking With Contraction	20
6.3 Funnel Control	20
7 Parameter Estimation and Model Fitting	21
7.1 System Identification for Multi-Body Models	21
7.2 Parameter Estimation Landscape	22
7.3 Muscle Parameter Calibration	23

8	Reinforcement Learning for Motor Control	24
8.1	RL Environments in the Companion Platform	24
8.2	Agent-Environment Loop	26
8.3	From Gym to Golf	26
9	Visualization and Analysis Tools	27
9.1	3D Motion Visualization	27
9.2	Jacobian and Manipulability Visualization	28
9.3	Energy Flow Diagrams	29
10	Capstone: Golf Swing Analysis	30
10.1	Model and Question	30
10.2	Geometry Before Attribution	30
10.3	What Adds at the Same State	31
10.4	A Counterfactual Is a New Trajectory	32
10.5	A Reproducible Synthetic Example	32
10.6	Connecting Mechanics, Control and Measurement	37

Nomenclature

General Conventions

- Scalars are lowercase or Greek letters (e.g., m, t, θ).
- Vectors are bold lowercase (e.g., $\mathbf{x}, \mathbf{u}, \mathbf{q}$).
- Matrices are bold uppercase (e.g., $\mathbf{A}, \mathbf{B}, \mathbf{M}, \mathbf{J}$).
- Expected values and sets are denoted by blackboard bold or script (e.g., $\mathbb{E}, \mathcal{S}$).

These are default conventions; each chapter declares its model, frames, and any local reuse of symbols. The mechanical entries below use n independent, regular local configuration coordinates and $\mathbf{v} = \dot{\mathbf{q}}$. This convention is local: it does not identify every configuration manifold with a Euclidean space or apply unchanged to redundant representations.

State and Kinematics

$\mathbf{q} \in \mathbb{R}^n$ Configuration coordinates in the declared local chart.

$\dot{\mathbf{q}} \in \mathbb{R}^n$ Coordinate rates; the generalized velocities under the local convention above.

$\ddot{\mathbf{q}} \in \mathbb{R}^n$ Second time derivatives of these coordinates, not necessarily Cartesian point accelerations.

$\mathbf{x} \in \mathbb{R}^{n_x}$ State-vector representation with n_x components; commonly $\mathbf{x} = [\mathbf{q}^T, \dot{\mathbf{q}}^T]^T$ locally, with actuator or internal states added when needed. Redundant state representations also retain their constraints.

$\mathbf{u} \in \mathbb{R}^m$ Control input vector (e.g., joint torques, muscle activations).

$\boldsymbol{\tau} \in \mathbb{R}^n$ Generalized forces conjugate to $\mathbf{q}$, with power $\boldsymbol{\tau}^T \dot{\mathbf{q}}$; components need not all be joint torques.

$t \in \mathbb{R}$ Time.

More generally, write $\mathbf{q} \in \mathbb{R}^{n_q}$ for a configuration representation and $\mathbf{v} \in \mathbb{R}^{n_v}$ for generalized velocities, with $\dot{\mathbf{q}} = \mathbf{N}(\mathbf{q})\mathbf{v}$ on the admissible domain. A unit quaternion has four components with a norm constraint, while angular velocity has three components in a declared frame. Configuration rates and generalized velocities therefore need not have the same dimension. See [Tedrake's multibody notes](#) for this representation convention. Constraints and any time-dependent kinematics must be included in the chosen model.

Inertia and Dynamics

M(**q**) Mass (inertia) matrix. It is symmetric positive definite for independent regular velocities with strictly positive modeled kinetic energy for every nonzero velocity. Constrained, redundant, massless, or singular representations require their own treatment.

C(**q**, $\dot{\mathbf{q}}$) Coriolis/centrifugal matrix; **C** $\dot{\mathbf{q}}$ is the generalized velocity-product force in the local mechanical equation. The matrix factorization is not unique.

g(**q**) Generalized gravity term; its sign follows the side of the mechanical equation on which the chapter places it.

J(**q**) Coordinate Jacobian of a time-independent task map **p**(**q**), so $\dot{\mathbf{p}} = \mathbf{J}\dot{\mathbf{q}}$ in the stated coordinates and frames.

With $\dot{\mathbf{q}} = \mathbf{N}\mathbf{v}$, the corresponding velocity Jacobian is $\mathbf{J}_v = \mathbf{J}_q\mathbf{N}$. Force components must change with the velocity convention: $\boldsymbol{\tau}_v = \mathbf{N}^T\boldsymbol{\tau}_q$ gives $\boldsymbol{\tau}_v^T\mathbf{v} = \boldsymbol{\tau}_q^T\dot{\mathbf{q}}$. Thus the same club-point motion and mechanical power must result from consistently paired kinematic and force representations.

Control and Optimization

For the indexed entries, declare a discrete update $\mathbf{x}_{k+1} = F_k(\mathbf{x}_k, \mathbf{u}_k)$ and a nominal pair $(\bar{\mathbf{x}}_k, \bar{\mathbf{u}}_k)$. Its derivatives are evaluated at that pair. These are derivatives of the full update, including the chosen sampling or numerical integration rule, not of zero-input drift alone.

A_k = $D_x F_k$ State Jacobian of the declared discrete update at the nominal pair.

B_k = $D_u F_k$ Input Jacobian of that update at the same pair.

V(**x**) or $\mathcal{V}$ Value function or Lyapunov function.

Q State penalty matrix in LQR/DDP.

R Control penalty matrix in LQR/DDP.

K Feedback gain matrix ($\delta\mathbf{u} = -\mathbf{K}\delta\mathbf{x}$).

γ A curve with a declared parameter; for specified initial data and an input law, $\gamma(t)$ denotes a particular timed state trajectory. A solution map $\phi(t; t_0, \mathbf{x}_0)$ instead retains dependence on initial time and state, for a fixed evolution law on its domain of existence.

Continuous-time chapters define their Jacobians from the full differential equation. Some use $f(\mathbf{x}, \mathbf{u}, t)$ for that full right-hand side; the autonomous $f(\mathbf{x})$ below denotes only the declared zero-input drift. Neither convention makes a discrete input Jacobian equal to the continuous one without accounting for the sampling interval and evolution.

Affine Control and Counterfactuals

$f(\mathbf{x})$ Drift vector field: complete autonomous evolution of the declared effective plant when $\mathbf{u} = \mathbf{0}$.

$G(\mathbf{x})$ Input (control) matrix field mapping control inputs to state derivatives.

$\dot{\mathbf{x}} = f(\mathbf{x}) + G(\mathbf{x})\mathbf{u}$ Control-affine system equations of motion.

$\mathbf{x}^{\text{ZTCF}}(t)$ Forward Zero-Torque Counterfactual trajectory under $\mathbf{u}(t) \equiv \mathbf{0}$.

$\mathbf{a}_{\text{ZVCF}}(\mathbf{q}, \mathbf{z})$ Instantaneous Zero-Velocity Counterfactual acceleration with velocity and declared control set to zero.

$\text{DCR}_{W,\mathcal{U}}(\mathbf{x})$ Drift-Control Ratio in a declared acceleration or task-projected space:

$$\text{DCR}_{W,\mathcal{U}}(\mathbf{x}) = \frac{\|W\mathbf{a}_{\text{drift}}\|_2}{\sup_{\mathbf{u} \in \mathcal{U}} \|WB_a\mathbf{u}\|_2 + \varepsilon}.$$

Biomechanics and Motor Control

$a_i \in [0, 1]$ Muscle activation level.

ℓ_i^M Muscle fiber length.

v_i^M Muscle fiber contraction velocity.

ℓ_i^{MT} Musculotendon unit length.

$\mathbf{F}_{\text{muscle}}$ Force generated by muscles.

$\mathbf{W}$ Muscle synergy matrix (weights).

$\mathbf{c}(t)$ Muscle synergy activation vector over time.

Geometry and Manifolds

$SE(3)$ Special Euclidean group of rigid body motions.

$SO(3)$ Special Orthogonal group of 3D rotations.

$\mathfrak{se}(3)$ Lie algebra of $SE(3)$, space of spatial twists (velocities).

$\mathcal{M}$ A smooth manifold.

$T_{\mathbf{x}}\mathcal{M}$ Tangent space at point $\mathbf{x}$.

Chapter 1

The Companion Platform

A simulation platform makes a modeling argument executable: which bodies move, which forces act, what the controller observes, and what an experiment measures. A common method name cannot by itself guarantee common physics. This chapter establishes contracts for turning the geometric and biological arguments of earlier volumes into a reproducible calculation. Its executable example is a simple pendulum, not a validated human swing or a demonstration of every adapter.

1.1 Architecture and Capability Boundaries

UpstreamDrift contains engine wrappers and shared Python interfaces. At the inspected revision, common interfaces live under `src/shared/python/engine_core/`, with engine implementations under `src/engines/physics_engines/`. The manager constructs an available engine through a registered factory. Loading, stepping, state access and dynamic calculations have distinct contracts. This is a source-code observation at a pinned revision, not proof that every method works for every model or installed backend [24].

Ask three separate questions: does the upstream engine offer an operation; does the wrapper implement it with the required conventions; and has that model-operation pair been tested? An activation-driven muscle environment cannot automatically answer an inverse-dynamics request for joint torques. A missing capability must give an explicit unsupported result, rather than a plausible array of zeros interpreted as physics.

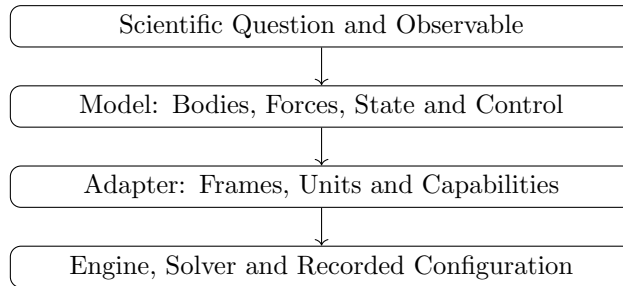


Figure 1.1: Contracts Connecting a Scientific Question to an Executable Model

1.1.1 What the Engine Names Mean

MuJoCo supplies multibody dynamics, actuators and soft contact. Drake provides multibody modeling and tools used in control and optimization; its contact configuration matters. Pinocchio supplies rigid-body algorithms, including recursive Newton–Euler inverse

dynamics (RNEA) and articulated-body forward dynamics (ABA). RNEA evaluates required forces from specified accelerations; ABA evaluates accelerations from specified forces. Tree-recursion complexity does not establish a real-time deadline for an entire application [15, 19, 1].

OpenSim represents musculoskeletal models and force components. MyoSuite is a collection of muscle-driven environments built on MuJoCo, not an OpenSim runtime; the provenance of a converted model differs from its current simulation backend [18, 16]. Choose an engine by required joints, forces, contacts, derivatives and available validation. A universal ranking such as “most accurate” or “best for golf” is not justified by this feature list.

1.2 State, Force and Time Contracts

Write the mechanical state as (q, v) , with

$$\dot{q} = N(q)v, \quad M(q)\dot{v} + c(q, v) = \tau_{\text{act}} + \tau_{\text{pass}} + J(q)^T \lambda. \quad (1.1)$$

Here c includes selected inertial and gravitational bias terms; passive forces must not also be counted in c if they appear on the right. A wrapper must declare what its “bias” output includes. Configurations and generalized velocities need not have equal array sizes: a unit quaternion has four stored components but describes three rotational degrees of freedom. Its derivative is not the angular-velocity vector [15].

The command u may denote joint torque, desired position, actuator input or muscle excitation. Activation and contraction dynamics can add state before force reaches the skeleton. Setting $u = 0$ is therefore not generally the same intervention as setting generalized torque to zero. Specify force frames, signs, units, timestamps and command holding rules before comparing a measured swing, a synthetic motion and a simulated release.

1.3 A Fully Specified Pendulum

Consider a uniform solid cylinder of mass m , length L and radius r , hinged at an end about an axis perpendicular to its length. Its COM is $L/2$ from the pivot. The transverse COM and hinge inertias are

$$I_C = \frac{m(L^2 + 3r^2)}{12}, \quad I_P = I_C + m(L/2)^2 = m \left(\frac{L^2}{3} + \frac{r^2}{4} \right). \quad (1.2)$$

The thin-rod limit is $mL^2/3$ about the pivot; a point bob at distance L would instead give mL^2 . These describe different mass distributions. Let $q = 0$ point vertically downward, positive rotation be about world $+y$, and gravity be world $-z$. With viscous hinge damping $b \geq 0$,

$$I_P \ddot{q} + b\dot{q} + mg(L/2) \sin q = 0, \quad (1.3)$$

$$E = \frac{1}{2} I_P \dot{q}^2 + mg(L/2)(1 - \cos q), \quad (1.4)$$

$$\dot{E} = -b\dot{q}^2. \quad (1.5)$$

Damping has units N m s/rad; energy is in joules. Defining $W_d(t) = \int_0^t b\dot{q}^2 ds$ gives the exact identity $E(t) + W_d(t) - E(0) = 0$. This is a numerical residual without dividing by $E(0)$, which is zero at rest in the downward configuration.

```

1 from dataclasses import dataclass
2
3 import numpy as np
4 from scipy.integrate import solve_ivp
5
6 GRAVITY = 9.81 # m/s^2; a declared example parameter
7
8
9 @dataclass(frozen=True)
10 class RodParameters:
11     """Positive solid-cylinder parameters; damping is nonnegative."""
12
13     mass: float = 1.0
14     length: float = 1.0
15     radius: float = 0.02
16     damping: float = 0.05
17
18     def __post_init__(self) -> None:
19         """Reject nonphysical parameters before constructing dynamics."""
20         values = np.array([self.mass, self.length,
21                             self.radius, self.damping])
22         if not np.all(np.isfinite(values)):
23             raise ValueError("parameters must be finite")
24         if np.any(values[:3] <= 0) or self.damping < 0:
25             raise ValueError("positive mass/dimensions; damping >= 0")
26
27     @property
28     def pivot_inertia(self) -> float:
29         """Transverse hinge inertia in kg m^2."""
30         return self.mass * (self.length**2 / 3 + self.radius**2 / 4)
31
32
33 def simulate_pendulum(parameters: RodParameters,
34                       initial: tuple[float, float],
35                       times: np.ndarray) -> dict[str, np.ndarray]:
36     """Integrate (angle, rate, dissipated work) on a requested output grid."""
37
38     times = np.asarray(times, dtype=float)
39     state = np.asarray(initial, dtype=float)
40     if (times.ndim != 1 or times.size < 2
41         or not np.all(np.isfinite(times))):
42         raise ValueError("times must be a finite 1D grid with >= 2 samples")
43     if times[0] != 0 or np.any(np.diff(times) <= 0):
44         raise ValueError("times must start at zero and strictly increase")
45     if state.shape != (2,) or not np.all(np.isfinite(state)):
46         raise ValueError("initial must contain finite angle and rate")
47     inertia = parameters.pivot_inertia
48     gravity_moment = parameters.mass * GRAVITY * parameters.length / 2
49
50     def derivative(time: float, values: np.ndarray) -> list[float]:
51         """Mechanical flow and cumulative dissipated work."""
52         angle, rate, _ = values
53         torque = -parameters.damping * rate - gravity_moment * np.sin(angle)
54         return [rate, torque / inertia, parameters.damping * rate**2]
55
56     solution = solve_ivp(
57         derivative, (0.0, times[-1]), [*state, 0.0], t_eval=times,
58         method="DOP853", rtol=1e-10, atol=1e-12,
59     )
60     if not solution.success:

```

```

60     raise RuntimeError("pendulum integration failed")
61     angle, rate, dissipated = solution.y
62     energy = 0.5 * inertia * rate**2 + gravity_moment * (1 - np.cos(angle))
63     return {"t": times, "q": angle, "qd": rate,
64            "energy": energy, "dissipated": dissipated}

```

Listing 1.1: Cylinder Pendulum with Explicit Energy Accounting

For an example call, use the default parameters, initial state (0.4,0), and 121 equally spaced output times from zero to three seconds. Energy includes the initial sample. This uses SciPy directly and does not invoke an UpstreamDrift adapter. Requested sample spacing is not the adaptive integrator’s internal step size. Check tolerance refinement and the energy-work residual; neither an energy check alone nor agreement with this idealized model validates human segment parameters.

1.4 Connecting the Example to the Swing

Segment inertia determines how torque changes angular acceleration; the configuration-dependent Jacobian maps motion to clubhead velocity. Damping dissipates energy, while constraints can redirect motion without supplying an independent energy source. Feedback changes future commands and the trajectory on which these quantities are evaluated. To study one contribution, preserve the remaining model and intervention assumptions explicitly. Chapter 3 encodes this pendulum as a model file; Chapter 2 explains comparisons that distinguish a different physical model from a different numerical method.

Exercises

1. **Energy and Work.** Evaluate $E + W_d - E(0)$ for damped and undamped motion. Tighten solver tolerances. Explain why decreasing E alone is not an integration-error measure when $b > 0$.
2. **Linear Limit.** Derive $\omega_0 = \sqrt{mgL/(2I_P)}$ for small angles. Compare with decreasing initial displacement and separate linearization error from numerical error.
3. **Integrator Choice.** Implement semi-implicit Euler, which updates velocity before position, and compare with a higher-order method using phase and work-balance errors. Do not infer exact energy conservation from the word “symplectic.”
4. **Adding a Link.** Declare the second link’s COM and inertia before deriving coupled dynamics. Investigate sensitivity to initial conditions for the chosen regime; a double pendulum is not chaotic on every trajectory.

Chapter 2

Physics Engine Comparison

An engine comparison needs a specified question, model and accuracy target. Solvers may disagree because their contact laws differ, an import changed the inertia, or an integration is inaccurate. Conversely, agreement can preserve a shared modeling mistake. This chapter gives a comparison protocol and executable error metric; it presents no measured cross-engine speed ranking or energy-drift curves.

2.1 Establish Model Equivalence First

Begin with the contact-free cylinder pendulum of Chapter 1. Match the fixed base, joint frame and axis, COM inertia, gravity, damping, initial state, force inputs and duration. Verify loaded parameters rather than assuming identical XML implies identical interpreted physics. At several states, compare mass, gravitational generalized force and passive torque against the analytic values I_P , $mgL \sin q/2$ and $-b\dot{q}$.

Next vary solver tolerances or internal step sizes and compare trajectories at common timestamps. An adaptive solver's output interval is not its internal step size. For fixed-step methods, record substeps and control-update timing. Interpolation introduces its own error, especially across impacts. Keep event-time and pre/post-impact comparisons separate from smooth-trajectory interpolation.

2.2 Measure the Right Error

For the unforced pendulum, define the energy-work residual

$$r_E(t) = E(t) - E(0) + \int_0^t b\dot{q}(s)^2 ds. \quad (2.1)$$

With applied generalized torque τ , subtract $\int_0^t \tau \dot{q} ds$ as well. Numerical integration of work contributes to residual error. With impacts or other dissipative forces, include the corresponding loss or jump law. A percentage relative to $E(0)$ is undefined when $E(0) = 0$ and sensitive to the arbitrary potential-energy reference; report joules or a declared positive characteristic scale instead.

Energy conservation does not guarantee phase accuracy. A trajectory can stay near the correct energy surface while reaching the wrong position at a specified time. For aligned Euclidean states x_i and reference states x_i^* , choose positive scales s_j with each coordinate's units and define

$$e_{ij} = \frac{x_{ij} - x_{ij}^*}{s_j}, \quad E_{\text{RMS}} = \left[\frac{1}{T} \int_{t_0}^{t_1} \|e(t)\|^2 dt \right]^{1/2}, \quad T = t_1 - t_0. \quad (2.2)$$

The scales make mixed angle/rate or position/velocity comparisons dimensionless and declare the relative importance assigned to each component. This code uses trapezoidal quadrature and reports the largest sampled norm, not a certified continuous-time maximum.

```

1 import numpy as np
2 from scipy.integrate import trapezoid
3
4
5 def trajectory_errors(times: np.ndarray, actual: np.ndarray,
6                       reference: np.ndarray,
7                       scales: np.ndarray) -> dict[str, float]:
8     """Return dimensionless RMS and sampled maximum normalized state errors."""
9
10    times = np.asarray(times, dtype=float)
11    actual = np.asarray(actual, dtype=float)
12    reference = np.asarray(reference, dtype=float)
13    scales = np.asarray(scales, dtype=float)
14    if (times.ndim != 1 or times.size < 2
15        or not np.all(np.isfinite(times))):
16        raise ValueError("times must be finite 1D with >= 2 samples")
17    if np.any(np.diff(times) <= 0):
18        raise ValueError("times must strictly increase")
19    if (actual.ndim != 2 or actual.shape != reference.shape
20        or actual.shape[0] != times.size or actual.shape[1] == 0):
21        raise ValueError("states must have matching nonempty (N, d) shapes")
22    if not np.all(np.isfinite(actual)) or not np.all(np.isfinite(reference)):
23        raise ValueError("states must be finite")
24    if (scales.shape != (actual.shape[1],)
25        or not np.all(np.isfinite(scales)) or np.any(scales <= 0)):
26        raise ValueError("one finite positive scale is required per
27        coordinate")
28    squared = np.sum(((actual - reference) / scales)**2, axis=1)
29    mean_square = trapezoid(squared, times) / (times[-1] - times[0])
30    return {"rms": float(np.sqrt(mean_square)),
31            "max": float(np.sqrt(np.max(squared)))}

```

Listing 2.1: Time-Weighted Errors on an Aligned Euclidean Grid

For $t = (0, 1, 3)$ and scalar normalized errors $(0, 1, 3)$, the trapezoidal integral of squared error is 10.5, giving $\sqrt{3.5}$. The unweighted node RMS is $\sqrt{10/3}$; neither equals the exact continuous RMS $\sqrt{3}$ for $e(t) = t$ on this coarse grid. Refinement matters.

The caller must first construct physically appropriate differences. Subtracting quaternion components is not an orientation error: q and $-q$ encode the same rotation. Use a declared relative-rotation logarithm and tangent-frame convention, with care near angle π . Periodic angles require a wrapping or unwrapping convention. Two simulations' difference is an agreement metric; calling it accuracy requires an independently justified reference, such as an analytic limit or a documented convergence study.

2.3 Contact Law, Solver and Integrator Are Separate Choices

The contact model specifies forces or impulses; the solver computes their coupled solution; the integrator advances state. Collision geometry and contact detection add further approximations. An engine name does not identify all these choices.

MuJoCo.

Its soft-contact formulation uses convex optimization and relaxes strict complementarity. Contact parameters describe the selected compliant behavior; integration method and step settings are separate choices. A nominal penetration value is neither automatically an error nor evidence of validated tissue deformation [15].

Drake.

Hydroelastic contact uses pressure fields and contact surfaces to model compliant interaction while bodies retain rigid-body motion. It does not resolve arbitrary elastic waves inside bodies. Geometry representations, compliant/rigid pairing and point-contact fallback settings affect which contacts are supported [1].

Pinocchio.

Constrained and impulse dynamics are available; saying “no contact dynamics” is incorrect. Supplying constraints differs from implementing collision detection, friction and time integration together. Identify the additional model and algorithms actually used [19].

OpenSim.

Behavior depends on the selected force component. For example, the documented HuntCrossleyForce uses Hertz-based interactions between spheres and half-spaces, with material and friction parameters. That component’s geometry restrictions do not describe every OpenSim contact component [18].

Changing contact stiffness changes the physical model and can also make its equations numerically harder to resolve. Report both effects. In golf, foot-ground loading, hand-club constraints and club-ball impact can require different resolutions. Agreement on a smooth pendulum does not validate these contact regimes.

2.4 A Reproducible Comparison Record

For each run record model and parameter hashes, engine and wrapper versions, gravity and force conventions, solver and integrator options, internal steps or tolerances, output and command timing, and initial state. Record computer, thread configuration, precision, initialization/warm-up policy and repeated timing trials. Separate loading, compilation, stepping and output costs. Report timing variability and failed runs, not just the fastest successful sample.

Compare cost at a declared error tolerance as well as error at a declared cost. Equal nominal timesteps need not yield equal accuracy. Start with analytic limits, then smooth multibody cases, and then the required contact regime. With experimental data, propagate measurement and parameter uncertainty: a converged solver cannot compensate for an incorrect joint axis or unmeasured external force. These checks support conclusions about a specified model and observable, not an unrestricted winner among engines.

Exercises

1. **Pendulum Refinement.** Use the same physical cylinder in two available implementations. Report parameter agreement, phase error, work residual and runtime across refinements; identify the reference’s own error.

2. **Nonuniform Sampling.** Reproduce the three-node example, refine the grid, and distinguish quadrature error from simulation error.
3. **Contact Reference.** Compare a bouncing-ball model with the ballistic flight and restitution law it assumes. Ideal elastic impact is a suitable reference only for that limit, not automatically for dissipative compliant contact.
4. **Swing Observable.** Choose clubhead velocity, joint torque or contact impulse. State units, alignment, uncertainty and model assumptions before choosing a metric.

Chapter 3

Building a Multibody Model

A model file encodes a hypothesis about geometry, mass and permitted motion. An attractive animation does not establish that these quantities are correct. Before building a whole golfer, construct a body whose kinematics and dynamics can be checked analytically. Then add anatomy and contacts while preserving the frames, parameters and question the model is meant to answer.

3.1 Formats and Their Boundaries

URDF means *Unified Robot Description Format*. It describes links and joints, including explicit frame transforms, inertia, and visual and collision geometry. Joint origins are not Denavit–Hartenberg parameters; convert and verify transforms when starting from a DH model. The standard urdfdom joint parser recognizes revolute, continuous, prismatic, fixed, floating and planar joints, not a type named `ball`. Individual importers may support only a subset [20, 25].

MJCF is MuJoCo’s distinct native XML format, not an extension of URDF syntax. It includes MuJoCo-specific actuator, tendon and contact descriptions. MuJoCo also imports URDF with documented conversion behavior and optional extensions; importing a rigid-body tree does not automatically transfer a complete muscle or controller model [14].

3.2 Frames, Inertia and Physical Consistency

Each joint needs a parent and child, its transform at zero configuration, and allowed motion. Every non-root link in a tree has one parent joint, and all links connect to a declared root. The word `world` is not an automatically declared link. Specify whether the importer fixes the root or adds a free base. Closed loops require additional constraints beyond a simple tree.

Mass, COM position and inertia must refer to compatible frames. If I_C is inertia about the COM in its principal frame, R rotates that frame into the reporting frame, and r runs from reporting point O to the COM, then

$$I_O = RI_C R^T + m[(r^T r)I_3 - rr^T]. \quad (3.1)$$

For nondegenerate three-dimensional bodies, COM inertia is symmetric positive definite. Principal moments also satisfy triangle inequalities, such as $I_1 + I_2 \geq I_3$; positive definiteness alone does not describe a physically possible mass distribution. Units are kg m^2 , not kg or m^2 . The numerical identity matrix is not a neutral inertia default.

URDF’s inertial origin locates the COM and its inertia frame; its six tensor entries are about that origin. Visual and collision origins are independent. In the parser, omitting an origin defaults to identity and omitting a joint axis defaults to $+x$. These defaults may parse

while describing the wrong mechanism. Revolute joints require limits; an inertial element needs both mass and inertia data [20, 25, 26].

3.3 A Complete Cylinder Pendulum

Use Chapter 1's parameters: $m = L = 1$ in SI units, $r = 0.02$ m and $b = 0.05$ N m s/rad. At zero angle, the cylinder extends from the hinge down world $-z$. Geometry and COM are centered at $z = -0.5$ m, and the hinge axis is $+y$. The transverse COM moments are 0.0834333333 kg m²; the axial moment is 0.0002 kg m². The fixed root below is a reference frame, not an extra moving mass.

```

1 <?xml version="1.0"?>
2 <robot name="cylinder_pendulum">
3   <link name="base"/>
4   <link name="rod">
5     <inertial>
6       <origin xyz="0 0 -0.5" rpy="0 0 0"/>
7       <mass value="1.0"/>
8       <inertia ixx="0.08343333333333333"
9               iyy="0.08343333333333333" izz="0.0002"
10              ixy="0" ixz="0" iyz="0"/>
11     </inertial>
12     <visual>
13       <origin xyz="0 0 -0.5" rpy="0 0 0"/>
14       <geometry><cylinder length="1.0" radius="0.02"/></geometry>
15     </visual>
16     <collision>
17       <origin xyz="0 0 -0.5" rpy="0 0 0"/>
18       <geometry><cylinder length="1.0" radius="0.02"/></geometry>
19     </collision>
20   </link>
21   <joint name="pivot" type="revolute">
22     <parent link="base"/>
23     <child link="rod"/>
24     <origin xyz="0 0 0" rpy="0 0 0"/>
25     <axis xyz="0 1 0"/>
26     <limit lower="-3.14" upper="3.14" effort="20" velocity="30"/>
27     <dynamics damping="0.05"/>
28   </joint>
29 </robot>

```

Listing 3.1: Complete URDF for the Analytic Cylinder Pendulum

Fix the base and set gravity to $(0, 0, -9.81)$ in the target engine. The analytic comparison assumes no contact or limit engagement. Limit and effort metadata do not prove every importer enforces those bounds identically. The file adds no actuator; the passive example needs none.

At angle q and rate v , the expected COM position and scalar dynamic terms are

$$p_C = \left(-\frac{1}{2} \sin q, 0, -\frac{1}{2} \cos q\right), \quad (3.2)$$

$$M = \frac{1}{3} + \frac{0.02^2}{4}, \quad g_q = 4.905 \sin q, \quad \tau_{\text{pass}} = -0.05v. \quad (3.3)$$

Here g_q is on the left in $M\dot{v} + g_q = \tau_{\text{pass}}$. These checks distinguish a correct XML tree from correct interpreted physics. Repository tests parse the actual listing and, when MuJoCo is installed, load it to check mass, coordinates, COM, inertia, gravity and damping.

That single-engine test does not certify import by Drake or Pinocchio, or a time-integrated comparison.

3.4 Extending to a Seven-Segment Golf Model

A possible reduced tree is a declared world reference, then pelvis, trunk, upper arm, forearm, hand, club shaft and club head. Connect the head to the shaft with a fixed joint; an orphan head loses its inertial contribution or causes import failure. For pelvis and trunk hinges, a three-DOF shoulder, elbow and wrist hinges, and fixed grip and head joints, the unconstrained mechanism has $1 + 1 + 3 + 1 + 1 = 7$ velocity degrees of freedom. A native quaternion shoulder gives eight stored configuration components with a unit-norm constraint; it does not create an eighth physical degree of freedom.

This describes one arm with a prescribed base support model. A second hand on the club generally adds a closed-loop constraint; legs and foot-ground interaction change the mechanism and external-force model. The seven-body sketch is not a whole-body golfer. Native spherical joints or a validated three-hinge decomposition are possible implementation choices. A decomposition has different coordinate singularities and intermediate-link requirements; zero-mass dummy links are not universally valid across engines.

For each segment, record parameter source, population or subject, anatomical landmarks, units, scaling rule, COM frame and inertia convention. Club properties need their own measurements or assumptions. Generic numbers labeled “an 80 kg golfer” are not an anthropometric calibration, and a partial upper-body chain need not sum to total body mass. Preserve parameter uncertainty and test its effect on the observable.

Muscle attachment points alone do not supply muscle mechanics. An extension needs paths and moment arms, activation and contraction states, force-length/velocity assumptions and parameter evidence. Frame errors affect Jacobians and moment arms; inertial errors affect required torques; unmodeled contacts affect inverse-dynamics residuals. All can change an apparent control strategy. Successful animation, exact inverse dynamics and an anatomically plausible explanation therefore represent different levels of evidence.

Exercises

1. **Frame Audit.** Perturb a joint origin, then an inertial origin. Predict which kinematic and dynamic checks change before running the model.
2. **Mass Distribution.** Keep mass and length fixed while changing cylinder radius. Derive changes in hinge inertia and small-angle frequency.
3. **Three-Link Arm.** Declare each frame, COM and tensor. Check tree connectivity, mass, positive kinetic energy and gravity before control.
4. **Golf Extension.** Specify constraints and external measurements needed for a second hand and ground contact. Distinguish measurements, illustrative assumptions and fitted quantities.

Chapter 4

Forward and Inverse Simulation

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

Forward simulation answers “what happens if...?”

Inverse dynamics answers “what caused this?”

The forward dynamics problem computes accelerations from forces, while inverse dynamics (also called the inverse kinematics problem when phrased in terms of finding required torques) computes forces from desired accelerations. Both are fundamental to control design and trajectory planning.

4.1 Forward Dynamics With the Companion Platform

Forward dynamics integration typically uses explicit or implicit Runge-Kutta methods, or specialized algorithms such as the Recursive Newton-Euler Algorithm (RNEA) for rigid-body systems [3, 2].

```
1 import numpy as np
2 from scipy.integrate import solve_ivp
3
4 def forward_dynamics_rnea(
5     q: np.ndarray,
6     qd: np.ndarray,
7     tau: np.ndarray,
8     model_params: dict
9 ) -> np.ndarray:
10     """Compute joint accelerations using RNEA-based forward dynamics.
11
12     Parameters
13     -----
14     q : np.ndarray, shape (n,)
15         Joint positions.
16     qd : np.ndarray, shape (n,)
17         Joint velocities.
18     tau : np.ndarray, shape (n,)
19         Applied joint torques.
20     model_params : dict
```

```

21     Model parameters (masses, lengths, inertias).
22
23     Returns
24     -----
25     np.ndarray, shape (n,)
26         Joint accelerations.
27     """
28     n = len(q)
29     masses = model_params['masses']
30     lengths = model_params['lengths']
31     g = model_params.get('gravity', 9.81)
32
33     # Simple 2-link model for demonstration
34     if n == 2:
35         m1, m2 = masses[:2]
36         l1, l2 = lengths[:2]
37         lc1, lc2 = l1/2, l2/2
38         I1 = m1 * l1**2 / 12
39         I2 = m2 * l2**2 / 12
40
41         # Mass matrix
42         M = np.array([
43             [I1 + I2 + m1*lc1**2 + m2*(l1**2 + lc2**2 + 2*l1*lc2*np.cos(q[1])
44             ),
45              [I2 + m2*(lc2**2 + l1*lc2*np.cos(q[1]))],
46              [I2 + m2*(lc2**2 + l1*lc2*np.cos(q[1])),
47              [I2 + m2*lc2**2]
48         ])
49
50         # Coriolis
51         h = m2 * l1 * lc2 * np.sin(q[1])
52         C = np.array([[ -h*qd[1], -h*(qd[0]+qd[1])],
53                       [ h*qd[0], 0]])
54
55         # Gravity
56         gvec = np.array([
57             (m1*lc1 + m2*l1)*g*np.sin(q[0]) + m2*lc2*g*np.sin(q[0]+q[1]),
58             m2*lc2*g*np.sin(q[0]+q[1])
59         ])
60
61         qdd = np.linalg.solve(M, tau - C @ qd - gvec)
62     else:
63         # Placeholder for general n-DOF
64         qdd = np.zeros(n)
65
66     return qdd
67
68 # Example simulation
69 params = {'masses': [2.0, 1.0], 'lengths': [0.3, 0.25], 'gravity': 9.81}
70 q0 = np.array([np.pi/4, np.pi/6])
71 qd0 = np.zeros(2)
72 qdd = forward_dynamics_rnea(q0, qd0, np.zeros(2), params)
73 print(f"Gravity-driven accelerations: {qdd}")

```

Listing 4.1: Forward dynamics simulation pipeline.

4.2 Simulation Loop and Integration Schemes

Figure 4.1 illustrates the typical forward simulation loop. At each timestep, we compute accelerations from the equations of motion, integrate to update velocities and positions, and advance to the next timestep.

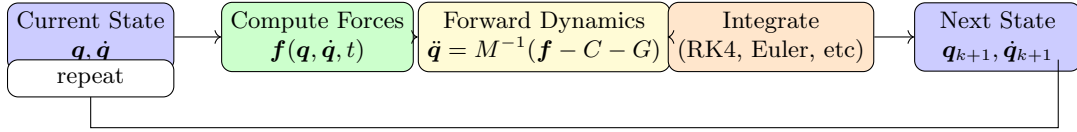


Figure 4.1: Forward simulation loop: from current state to forces (gravity, contacts, actuators), compute accelerations via forward dynamics, integrate, and update to next state.

4.3 Inverse Dynamics Pipeline

The inverse dynamics problem solves for torques given desired motion: given $\mathbf{q}(t), \dot{\mathbf{q}}(t), \ddot{\mathbf{q}}(t)$, compute $\boldsymbol{\tau}(t)$. This is implemented via the Recursive Newton-Euler Algorithm (RNEA) [3, 7]. The RNEA can be run either forward (for forward dynamics) or backward (for inverse dynamics) through the kinematic chain.

Exercises

1. Run forward dynamics for a 2-link pendulum and plot trajectories.
2. Implement inverse dynamics and verify $\text{ID}(\text{FD}(\boldsymbol{\tau})) = \boldsymbol{\tau}$.
3. Compare with Pinocchio's RNEA implementation for timing.

Chapter 5

Trajectory Optimization in Practice

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

Theory without practice is empty.
Practice without theory is blind.

5.1 DDP/iLQR Implementation

Differential Dynamic Programming (DDP) and iterative Linear Quadratic Regulator (iLQR) are related algorithms for trajectory optimization [13, 4]. They solve the finite-horizon optimal control problem by iteratively linearizing around a nominal trajectory and solving quadratic subproblems. Figure 5.1 shows the algorithm structure.

Here we implement iLQR end-to-end:

```
1 import numpy as np
2 from typing import Callable, Tuple
3
4 def ilqr_solve(
5     dynamics: Callable,
6     cost_running: Callable,
7     cost_terminal: Callable,
8     x0: np.ndarray,
9     u_init: np.ndarray,
10    n_iterations: int = 50,
11    regularization: float = 1e-6
12 ) -> Tuple[np.ndarray, np.ndarray, list[float]]:
13     """Iterative Linear Quadratic Regulator.
14
15     Parameters
16     -----
17     dynamics : callable
18         (x, u) -> x_next, (A, B) Jacobians
19     cost_running : callable
20         (x, u) -> (cost, lx, lu, lxx, luu, lux)
21     cost_terminal : callable
22         x -> (cost, lx, lxx)
```

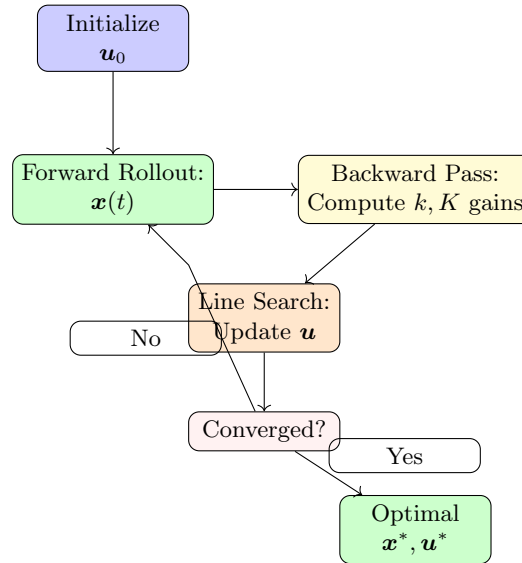


Figure 5.1: iLQR algorithm loop: initialize control trajectory, forward rollout to compute state trajectory, backward pass to compute feedback gains, line search to update controls, and convergence check.

```

23     x0 : np.ndarray
24         Initial state.
25     u_init : np.ndarray, shape (T, m)
26         Initial control trajectory.
27     n_iterations : int
28         Maximum iterations.
29     regularization : float
30         Regularization for Quu inversion.
31
32     Returns
33     -----
34     Tuple
35         (x_traj, u_traj, costs)
36     """
37     T, m = u_init.shape
38     n = len(x0)
39
40     # Forward rollout with initial controls
41     x_traj = np.zeros((T + 1, n))
42     x_traj[0] = x0
43     u_traj = u_init.copy()
44
45     for t in range(T):
46         x_next, _ = dynamics(x_traj[t], u_traj[t])
47         x_traj[t + 1] = x_next
48
49     costs = []
50
51     for iteration in range(n_iterations):
52         # Compute total cost
53         total_cost = sum(

```



```

54         cost_running(x_traj[t], u_traj[t])[0]
55         for t in range(T)
56     ) + cost_terminal(x_traj[T])[0]
57     costs.append(total_cost)
58
59     # Backward pass
60     Vx_next, Vxx_next = cost_terminal(x_traj[T])[1:]
61     k_gains = np.zeros((T, m))
62     K_gains = np.zeros((T, m, n))
63
64     for t in range(T - 1, -1, -1):
65         _, (A, B) = dynamics(x_traj[t], u_traj[t])
66         _, lx, lu, lxx, luu, lux = cost_running(
67             x_traj[t], u_traj[t]
68         )
69
70         Qx = lx + A.T @ Vx_next
71         Qu = lu + B.T @ Vx_next
72         Qxx = lxx + A.T @ Vxx_next @ A
73         Quu = luu + B.T @ Vxx_next @ B + regularization * np.eye(m)
74         Qux = lux + B.T @ Vxx_next @ A
75
76         Quu_inv = np.linalg.inv(Quu)
77         k_gains[t] = -Quu_inv @ Qu
78         K_gains[t] = -Quu_inv @ Qux
79
80         Vx_next = Qx + K_gains[t].T @ Quu @ k_gains[t]
81         Vxx_next = Qxx + K_gains[t].T @ Quu @ K_gains[t]
82
83     # Forward pass with line search
84     alpha = 1.0
85     x_new = np.zeros_like(x_traj)
86     u_new = np.zeros_like(u_traj)
87     x_new[0] = x0
88
89     for t in range(T):
90         dx = x_new[t] - x_traj[t]
91         u_new[t] = u_traj[t] + alpha * k_gains[t] + K_gains[t] @ dx
92         x_new[t + 1], _ = dynamics(x_new[t], u_new[t])
93
94     x_traj = x_new
95     u_traj = u_new
96
97     if iteration > 0 and abs(costs[-1] - costs[-2]) < 1e-6:
98         break
99
100     return x_traj, u_traj, costs

```

Listing 5.1: iLQR implementation for trajectory optimization.

5.2 Application: Optimal Pendulum Swing-Up

The classic control benchmark: swing a pendulum from the hanging position to the inverted equilibrium.

5.3 Application: Optimal Pendulum Swing-Up

The pendulum swing-up is a classic trajectory optimization benchmark. The goal is to find the minimum-energy torque sequence that brings the pendulum from the downward (stable) equilibrium to the upright (unstable) equilibrium.

5.4 Application: Golf Swing Optimization

DDP can be applied to a simplified golf model (3-5 DOF) to optimize clubhead speed at ball impact. The cost function would penalize control effort while maximizing endpoint velocity. In practice, golf swing optimization requires careful handling of contact constraints at impact.

Exercises

1. Use the iLQR code to solve the pendulum swing-up problem.
2. Extend to a double pendulum and optimize for maximum tip speed.
3. Add muscle force constraints and solve the muscle-driven swing-up.

Chapter 6

Controller Design and Testing

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

A controller is only as good as its worst-case performance.

This chapter covers implementation of feedback controllers for multi-body systems, from proportional-derivative (PD) control to contraction-based designs. All controllers assume full state feedback, i.e., direct access to positions and velocities.

6.1 PD Control With Gravity Compensation

The proportional-derivative controller with gravity compensation is the simplest nonlinear controller that guarantees asymptotic stability for mechanical systems without external disturbances [22].

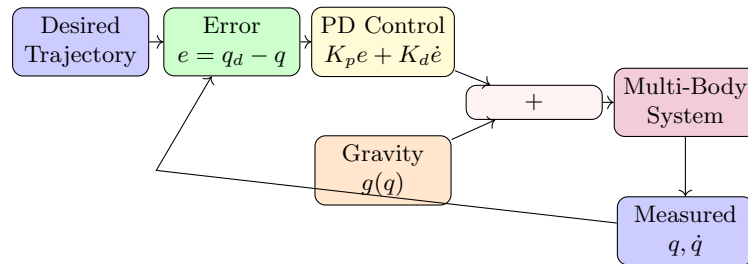


Figure 6.1: PD control with gravity compensation. The controller computes an error signal from desired and measured positions, applies PD gains, and adds gravity compensation to achieve asymptotic tracking.

```
1 import numpy as np
2
3 class PDGravityCompController:
4     """PD controller with gravity compensation.
5
6     From Volume I, Chapter 3: the simplest nonlinear controller
```

```

7  that guarantees asymptotic stability.
8  """
9
10 def __init__(self, Kp: np.ndarray, Kd: np.ndarray,
11               gravity_func=None):
12     self.Kp = Kp
13     self.Kd = Kd
14     self.gravity_func = gravity_func
15
16 def compute(self, q: np.ndarray, qd: np.ndarray,
17            q_des: np.ndarray, qd_des: np.ndarray) -> np.ndarray:
18     """Compute control torques.
19
20     Parameters
21     -----
22     q, qd : current state
23     q_des, qd_des : desired state
24
25     Returns
26     -----
27     tau : np.ndarray, control torques
28     """
29     tau = self.Kp @ (q_des - q) + self.Kd @ (qd_des - qd)
30     if self.gravity_func is not None:
31         tau += self.gravity_func(q)
32     return tau

```

Listing 6.1: PD controller with gravity compensation.

6.2 Trajectory Tracking With Contraction

Contraction theory provides tools for analyzing nonlinear stability with global guarantees [6, 12]. A contraction-based controller designs the feedback law to make the closed-loop system contracting with respect to a suitable metric, guaranteeing exponential convergence to the desired trajectory from any initial condition.

6.3 Funnel Control

Trajectory funnels define regions around a nominal trajectory within which the controller guarantees convergence [11]. This is useful for handling model uncertainty and disturbances. Funnel computation requires solving a set of linear matrix inequalities (LMIs) at each time step.

Exercises

1. Implement PD control for the 2-DOF arm and track a circular trajectory.
2. Add gravity compensation and compare tracking error.
3. Implement a contraction-based controller and verify convergence.

Chapter 7

Parameter Estimation and Model Fitting

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

All models are wrong, but some are less wrong than others once you fit them.

Parameter estimation uses measured motion data to infer unknown model parameters such as segment masses, inertias, and damping coefficients. This chapter covers least-squares methods for linear and bilinear problems, and optimization methods for nonlinear problems.

7.1 System Identification for Multi-Body Models

The linear regression approach exploits the linearity of the equations of motion with respect to inertial parameters. Given measured trajectories $\mathbf{q}(t)$, $\dot{\mathbf{q}}(t)$, $\ddot{\mathbf{q}}(t)$ and applied torques $\boldsymbol{\tau}(t)$, we form the regressor matrix $Y(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}})$ such that

$$\boldsymbol{\tau} = Y(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}})\boldsymbol{\pi} \quad (7.1)$$

where $\boldsymbol{\pi}$ is the parameter vector [7]. The least-squares solution minimizes $\|Y\boldsymbol{\pi} - \boldsymbol{\tau}\|_2^2$.

```
1 import numpy as np
2 from scipy.optimize import minimize
3
4 def estimate_inertial_params(
5     q_data: np.ndarray,
6     qd_data: np.ndarray,
7     qdd_data: np.ndarray,
8     tau_data: np.ndarray,
9     n_params: int
10 ) -> np.ndarray:
11     """Estimate inertial parameters using least-squares.
12
13     The equations of motion are LINEAR in inertial parameters:
14     tau = Y(q, qd, qdd) * pi
15     where Y is the regressor matrix and pi is the parameter vector.
```

```

16
17     Parameters
18     -----
19     q_data : (N, n) joint positions
20     qd_data : (N, n) joint velocities
21     qdd_data : (N, n) joint accelerations
22     tau_data : (N, n) joint torques
23     n_params : int number of inertial parameters
24
25     Returns
26     -----
27     np.ndarray estimated parameters
28     """
29     N, n = q_data.shape
30
31     # Build regressor matrix (simplified for 1-DOF)
32     Y = np.zeros((N * n, n_params))
33     tau_vec = tau_data.reshape(-1)
34
35     for k in range(N):
36         for j in range(n):
37             row = k * n + j
38             Y[row, 0] = qdd_data[k, j] # Inertia
39             Y[row, 1] = np.sin(q_data[k, j]) # Gravity
40
41     # Least squares solution
42     params, _, _, _ = np.linalg.lstsq(Y, tau_vec, rcond=None)
43     return params

```

Listing 7.1: Inertial parameter estimation from motion data.

7.2 Parameter Estimation Landscape

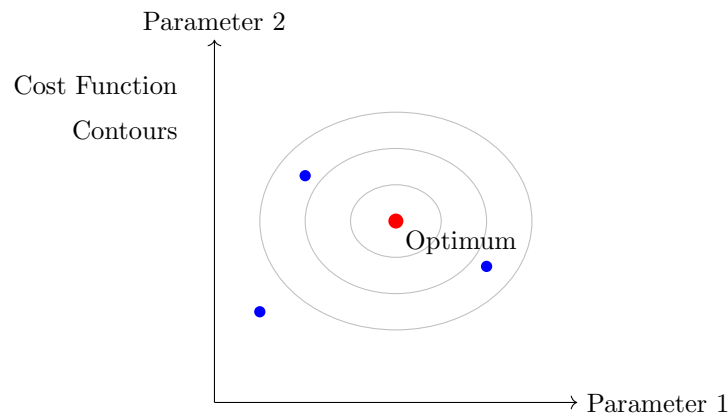


Figure 7.1: Parameter estimation as optimization over the parameter space. Least-squares methods seek the minimum of the cost function (red point) from initial guesses (blue points).

7.3 Muscle Parameter Calibration

Muscle parameters (Hill model constants: a , $F_{\max}$, optimal fiber length, etc.) can be estimated by fitting model predictions to force or EMG measurements. This requires nonlinear optimization, typically using gradient-based methods [17].

Exercises

1. Estimate the mass and length of a pendulum from simulated motion data.
2. Add measurement noise and study estimation robustness.
3. Implement Bayesian estimation with prior knowledge from literature.

Chapter 8

Reinforcement Learning for Motor Control

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

When the model is too complex for optimal control, let the agent learn.

Reinforcement learning (RL) methods can solve motor control problems when the cost function is difficult to specify or when model uncertainty is large [23]. This chapter demonstrates how to formulate multi-body control tasks as RL environments and integrate with standard policy optimization algorithms like PPO [21].

8.1 RL Environments in the Companion Platform

The companion platform provides Gymnasium-compatible environments for motor control tasks [23]. Gymnasium (formerly OpenAI Gym) is a standard interface for defining RL environments with reset, step, and reward computation methods. This enables use of any RL algorithm implemented for Gymnasium.

```
1 import numpy as np
2
3 class PendulumEnv:
4     """Simple pendulum environment for RL benchmarks.
5
6     State: [cos(theta), sin(theta), theta_dot]
7     Action: torque (continuous, clipped to [-2, 2])
8     Reward: -(angle^2 + 0.1*vel^2 + 0.01*action^2)
9     """
10
11     def __init__(self, dt: float = 0.05, max_steps: int = 200):
12         self.dt = dt
13         self.max_steps = max_steps
14         self.g = 9.81
15         self.l = 1.0
16         self.m = 1.0
17         self.max_torque = 2.0
18         self.state = np.zeros(2)
```



```

19         self.step_count = 0
20
21     def reset(self, seed: int | None = None) -> np.ndarray:
22         if seed is not None:
23             np.random.seed(seed)
24         self.state = np.array([
25             np.pi + np.random.uniform(-0.1, 0.1),
26             np.random.uniform(-0.5, 0.5)
27         ])
28         self.step_count = 0
29         return self._get_obs()
30
31     def _get_obs(self) -> np.ndarray:
32         theta, theta_dot = self.state
33         return np.array([np.cos(theta), np.sin(theta), theta_dot])
34
35     def step(self, action: float) -> tuple:
36         theta, theta_dot = self.state
37         u = np.clip(action, -self.max_torque, self.max_torque)
38
39         # Dynamics
40         theta_ddot = (-self.g / self.l * np.sin(theta) +
41                      u / (self.m * self.l**2))
42         theta_dot += theta_ddot * self.dt
43         theta += theta_dot * self.dt
44
45         self.state = np.array([theta, theta_dot])
46         self.step_count += 1
47
48         # Reward
49         angle_normalized = ((theta + np.pi) % (2*np.pi)) - np.pi
50         reward = -(angle_normalized**2 +
51                  0.1 * theta_dot**2 +
52                  0.01 * u**2)
53
54         done = self.step_count >= self.max_steps
55         return self._get_obs(), reward, done, {}
56
57
58 # Quick test
59 env = PendulumEnv()
60 obs = env.reset(seed=42)
61 total_reward = 0.0
62 for _ in range(200):
63     action = np.random.uniform(-2, 2)
64     obs, reward, done, _ = env.step(action)
65     total_reward += reward
66     if done:
67         break
68 print(f"Random policy total reward: {total_reward:.1f}")

```

Listing 8.1: Custom RL environment for pendulum control.

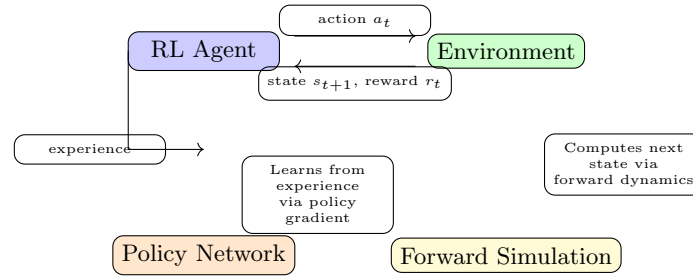


Figure 8.1: RL agent-environment loop. The agent selects actions via its policy network; the environment integrates dynamics and computes rewards. Both interact repeatedly.

8.2 Agent-Environment Loop

8.3 From Gym to Golf

The companion platform’s RL pipeline extends from simple benchmarks (pendulum, reaching) to full muscle-driven golf swing optimization using MyoSuite environments. The reward function is task-specific; for golf, one might maximize clubhead speed at impact while penalizing muscle effort.

Exercises

1. Train a PPO agent to swing up the pendulum. Plot the learning curve.
2. Compare RL-learned policy with the iLQR solution from Chapter 5.
3. Design a reward function for maximizing clubhead speed in a golf model.

Chapter 9

Visualization and Analysis Tools

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

If you cannot see it, you cannot understand it.

9.1 3D Motion Visualization

Visualization of multi-body trajectories is essential for understanding simulated motion and validating controller behavior. This section covers kinematics-based visualization (stick figures), manipulability analysis, and energy flow diagrams.

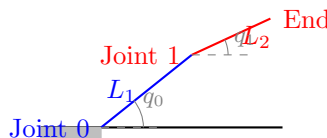


Figure 9.1: Stick-figure representation of a 2-link arm. Visualization code computes joint positions from angles using forward kinematics, then renders links and joints.

```
1 import numpy as np
2
3 def compute_joint_positions_2d(
4     q: np.ndarray,
5     lengths: list[float]
6 ) -> np.ndarray:
7     """Compute 2D joint positions for a serial chain.
8
9     Parameters
10    -----
11    q : np.ndarray, shape (n,)
12        Joint angles.
13    lengths : list[float]
```

```

14         Segment lengths.
15
16     Returns
17     -----
18     np.ndarray, shape (n+1, 2)
19         XY positions of each joint (including base).
20     """
21     n = len(q)
22     positions = np.zeros((n + 1, 2))
23     angle_cumulative = 0.0
24
25     for i in range(n):
26         angle_cumulative += q[i]
27         positions[i + 1, 0] = (positions[i, 0] +
28                               lengths[i] * np.cos(angle_cumulative))
29         positions[i + 1, 1] = (positions[i, 1] +
30                               lengths[i] * np.sin(angle_cumulative))
31
32     return positions
33
34
35 def phase_portrait(
36     q_history: np.ndarray,
37     qd_history: np.ndarray,
38     joint_idx: int = 0,
39     label: str = "Joint 0"
40 ) -> dict:
41     """Compute phase portrait data for a single joint.
42
43     Parameters
44     -----
45     q_history : (N, n) trajectory data
46     qd_history : (N, n) velocity data
47     joint_idx : int
48     label : str
49
50     Returns
51     -----
52     dict with 'x', 'y', 'label' keys
53     """
54     return {
55         'x': q_history[:, joint_idx],
56         'y': qd_history[:, joint_idx],
57         'label': label
58     }

```

Listing 9.1: Stick-figure animation for multi-body motion.

9.2 Jacobian and Manipulability Visualization

Visualizing the velocity manipulability ellipsoid JJ^T at each configuration [5]. The Jacobian maps joint velocities to end-effector velocities: $\mathbf{v}_{ee} = J(\mathbf{q})\dot{\mathbf{q}}$. The manipulability ellipsoid is the image of the unit ball under J , and its principal axes show the directions in which the end-effector can move with maximum ease.

9.3 Energy Flow Diagrams

Energy flow diagrams track how kinetic and potential energy change during motion, and how energy is transferred between segments via the kinetic chain. For a multi-segment system, the total energy is

$$E_{\text{total}} = \sum_i (KE_i + PE_i) = \sum_i \left(\frac{1}{2} I_i \dot{q}_i^2 + m_i g h_i \right) \quad (9.1)$$

where I_i is the effective inertia, $\dot{q}_i$ is the angular velocity, and h_i is the height of the center of mass of segment i . During ballistic phases (no muscle torque), energy is conserved and transferred between kinetic and potential forms.

Exercises

1. Animate a 3-link arm tracking a circular path.
2. Plot the manipulability ellipsoid during a reaching movement.
3. Create an energy flow diagram for the double pendulum golf model.

Chapter 10

Capstone: Golf Swing Analysis

Draft Notice. This chapter provides a runnable, tested three-link example and a roadmap for larger investigations. Its synthetic motion is prescribed, not measured or optimized. The extensions on muscles, feedback, uncertainty and impact require additional implementation and evidence.

The scientific question is how geometry, inertia and actuation together produce an end-point motion. A rapidly moving clubhead does not by itself reveal which joint supplied energy, which muscle was active, or what would happen after removing an input. Those questions require different calculations. This capstone connects them through one explicit mechanical model, then identifies the extra information each extension needs.

10.1 Model and Question

The implemented system has three rigid links and three revolute coordinates. The base is fixed. The first angle is measured counterclockwise from the horizontal; the other two are relative joint angles. World height increases upward and gravity acts downward. Angles, rates and accelerations use radians, rad/s and rad/s²; lengths, masses and generalized torques use metres, kilograms and Nm. The chain can illustrate a shoulder–elbow–wrist linkage, but it has no moving torso, two-arm closed loop, muscles, shaft flexibility, ground contact or ball collision.

Each link is a uniform rod with centre of mass at half its length and central inertia $I_i = m_i \ell_i^2 / 12$. The parallel-axis contributions are included when the model assembles its mass matrix. The default values are illustrative parameters, not subject-specific measurements. A head-heavy club requires measured or justified centre-of-mass and inertia parameters; changing its mass alone does not implement that distribution. The model therefore supports conditional mechanical deductions, not a prescription for human technique.

10.2 Geometry Before Attribution

Let $\theta_i = \sum_{j=1}^i q_j$ and $\omega_i = \sum_{j=1}^i \dot{q}_j$. The tip position and velocity are

$$\mathbf{p}(q) = \sum_{i=1}^3 \ell_i \begin{bmatrix} \cos \theta_i \\ \sin \theta_i \end{bmatrix}, \quad (10.1)$$

$$\mathbf{v}(q, \dot{q}) = \sum_{i=1}^3 \ell_i \omega_i \begin{bmatrix} -\sin \theta_i \\ \cos \theta_i \end{bmatrix} = J_p(q) \dot{q}. \quad (10.2)$$

Here J_p is the 2×3 Jacobian of the tip position. It is not the six-component spatial-twist Jacobian; the distinction matters when comparing robotics conventions [10]. Speed is $s_v = \|\mathbf{v}\|$, taken *after* adding the velocity vectors. Their directions depend on configuration. The scalar sum $\sum_i \ell_i |\omega_i|$ discards directional cancellation.

For the default lengths (0.28, 0.46, 1.10) m and relative rates (5, 5, 5) rad/s, the straight configuration $q = (0, 0, 0)$ gives 22.5 m/s. At $q = (0, \pi, \pi)$ the signed transverse contributions are $1.4 - 4.6 + 16.5 = 13.3$ m/s. The configuration-blind value would overestimate this particular folded example by about 69%. This is a reproducible geometric counterexample, not a population estimate.

10.3 What Adds at the Same State

For this unconstrained rigid model, write

$$M(q)\ddot{q} + b(q, \dot{q}) = \tau, \quad b = C(q, \dot{q})\dot{q} + g(q), \quad M(q) \succ 0. \quad (10.3)$$

Forward dynamics evaluates acceleration from the state and input; inverse dynamics evaluates the input required by a specified acceleration [9]. At a *common* state define

$$a_d = -M^{-1}b, \quad a_u = M^{-1}\tau, \quad \ddot{q} = a_d + a_u. \quad (10.4)$$

These are signed joint-acceleration vectors. The inverse is notation: the code solves linear systems. The equality is exact in the mathematical model, subject to numerical error in its implementation. It does not establish that arbitrary supplied position, velocity and torque arrays form a dynamically consistent trajectory.

Common Misconception

The magnitudes $\|a_d\|$ and $\|a_u\|$ generally do not add to $\|\ddot{q}\|$. If $\tau = b \neq 0$, then $a_u = -a_d$ and the net joint acceleration is zero, even though both component norms are positive. Reporting those norms as complementary percentages loses the cancellation. The norms also depend on the chosen joint coordinates and weighting. They are neither a speed partition nor a measure of muscular effort.

The tip acceleration contains an additional geometric term:

$$\mathbf{a}_p = J_p \ddot{q} + \dot{J}_p \dot{q} = \mathbf{a}_{p,d} + \mathbf{a}_{p,u}, \quad (10.5)$$

$$\mathbf{a}_{p,d} = J_p a_d + \dot{J}_p \dot{q}, \quad \mathbf{a}_{p,u} = J_p a_u, \quad (10.6)$$

$$\dot{J}_p \dot{q} = - \sum_{i=1}^3 \ell_i \omega_i^2 \begin{bmatrix} \cos \theta_i \\ \sin \theta_i \end{bmatrix}. \quad (10.7)$$

These vectors have units m/s². When $s_v > 0$, its instantaneous rate is

$$\dot{s}_v = \frac{\mathbf{v}^\top \mathbf{a}_{p,d}}{s_v} + \frac{\mathbf{v}^\top \mathbf{a}_{p,u}}{s_v}. \quad (10.8)$$

The two signed tangential terms can oppose one another. This formula is undefined at zero speed; acceleration vectors remain meaningful there. The current listing reports joint components, not these Cartesian projections.

At a straight vertical configuration, the ideal model has $g = 0$ and $C\dot{q} = 0$: derivatives of its relative-angle cosine terms vanish. Nevertheless, $\dot{J}_p\dot{q}$ can be nonzero. A rotating straight chain can have zero joint drift acceleration at that instant and a substantial centripetal tip acceleration. Straightness alone is insufficient even for zero joint drift: a horizontal chain generally has nonzero gravity torque.

10.4 A Counterfactual Is a New Trajectory

With $x = (q, \dot{q})$, write $\dot{x} = f(x) + G(x)\tau$. A zero-torque trajectory released at t_r obeys

$$\dot{x}_0 = f(x_0), \quad x_0(t_r) = x_u(t_r). \quad (10.9)$$

For a driven trajectory x_u , their difference satisfies

$$\frac{d}{dt}(x_u - x_0) = f(x_u) - f(x_0) + G(x_u)\tau. \quad (10.10)$$

The drift difference vanishes at release but generally not afterward. Thus integrating $M^{-1}\tau$ along the driven path does not by itself recover the difference in subsequent joint velocities. Different configurations also change the tip Jacobian. Control-affine input dependence does not make entire trajectories, or their speed magnitudes, superposable.

A useful comparison declares the release state, retained passive forces, intervention and endpoint: a common elapsed time and each trajectory's contact event are different experiments. Turning off excitation in a muscle model is also different from setting generalized torque to zero; activation and tendon states can retain history. Here the unforced system retains gravity and inertia and contains no friction or elastic elements. Energy conservation and timestep refinement provide checks on its numerical integration [8]. They do not validate its human anatomy. The separate `GolfModel.ztcf_trajectory` routine can support an unforced extension; `SwingAnalysis` does not call it.

10.5 A Reproducible Synthetic Example

The listing prescribes a smooth path, evaluates its analytic first and second derivatives, and calculates $\tau = M\ddot{q} + C\dot{q} + g$. These are required model torques for that path, with no imposed actuator limits. They replace an unrelated illustrative torque profile. Inverse-dynamic consistency tests an algebraic relationship; a separate forward integration and step-refinement comparison would test trajectory reproduction.

For duration $T > 0$ and maximum sample spacing $h > 0$, the grid contains $\lceil T/h \rceil + 1$ samples including both endpoints. Its actual spacing is $T/\lceil T/h \rceil$, which need not equal h . Analytic rates avoid confusing those two intervals. With $\sigma = t/T$, the prescribed path is

$$q(\sigma) = \begin{bmatrix} -\pi/2 + \pi\sigma^2 \\ (\pi/3)\sin(\pi\sigma) \\ -(\pi/2)\cos(\pi\sigma/2) \end{bmatrix}. \quad (10.11)$$

Its nonzero endpoint rates are allowed; this example is not a rest-to-rest trajectory. Doubling T at fixed path halves rates and quarters accelerations; gravity torque does not scale that way. This distinguishes changing the path, changing its timing and changing the mechanical plant.


```

1  """Same-state analysis of a prescribed three-link planar motion.
2
3  The generated Volume V listing uses this tested module. It computes tip speed
4  and instantaneous joint-acceleration components, not counterfactual
5  trajectories,
6  muscle forces or an optimized human swing. Synthetic torques are inverse-
7  dynamic
8  requirements of the prescribed motion under the uniform-rod model.
9  """
10
11 from __future__ import annotations
12
13 from dataclasses import dataclass, field
14 from typing import cast
15
16 import numpy as np
17 from numpy.typing import NDArray
18
19 from src.affine_control.golf_model import GolfModel
20
21 __all__ = ["SwingAnalysis", "synthetic_swing"]
22
23 type Array = NDArray[np.float64]
24
25 METRES_PER_SECOND_TO_MPH = 2.236936
26
27 def _rod_inertia(mass: float, length: float) -> float:
28     """Moment of inertia of a uniform rod about its own centre of mass."""
29     return mass * length**2 / 12.0
30
31
32 def _finite_array(value: Array, shape: tuple[int, ...], name: str) -> Array:
33     """Check shape and finiteness at the mutable analysis boundary."""
34     array = np.asarray(value, dtype=float)
35     if array.shape != shape or not np.all(np.isfinite(array)):
36         raise ValueError(f"{name} must be finite with shape {shape}")
37     return array
38
39
40 @dataclass
41 class SwingAnalysis:
42     """Kinematic and dynamic analysis of a three-link planar swing.
43
44     The chain is shoulder, elbow, wrist, carrying the club as the distal link
45     .
46     Angles are relative joint angles; the model treats each segment as a
47     uniform
48     rod, so its inertia follows from mass and length rather than being a free
49     parameter.
50     """
51
52     segment_lengths: Array = field(default_factory=lambda: np.array([0.28,
53     0.46, 1.10]))
54     segment_masses: Array = field(default_factory=lambda: np.array([2.16,
55     1.79, 0.30]))
56
57     time: Array | None = None
58     joint_angles: Array | None = None

```

```

55 joint_velocities: Array | None = None
56 joint_torques: Array | None = None
57
58 def model(self) -> GolfModel:
59     """The rigid-body model implied by the segment parameters."""
60     lengths_array = _finite_array(self.segment_lengths, (3,), "
segment_lengths")
61     masses_array = _finite_array(self.segment_masses, (3,), "
segment_masses")
62     if np.any(lengths_array <= 0):
63         raise ValueError("segment_lengths must be positive")
64     if np.any(masses_array <= 0):
65         raise ValueError("segment_masses must be positive")
66     lengths = tuple(map(float, lengths_array))
67     masses = tuple(map(float, masses_array))
68     inertias = tuple(_rod_inertia(m, ell) for m, ell in zip(masses,
lengths, strict=True))
69     return GolfModel(masses=masses, lengths=lengths, inertias=inertias)
70     # type: ignore[arg-type]
71
72 def _require_kinematics(self) -> tuple[Array, Array, Array]:
73     """Return '(time, angles, rates)', or fail with a usable message."
74     """
75     if self.time is None or self.joint_angles is None or self.
joint_velocities is None:
76         raise ValueError("set time, joint_angles and joint_velocities
before analysing")
77     time = np.asarray(self.time, dtype=float)
78     if time.ndim != 1 or time.size == 0 or not np.all(np.isfinite(time)):
79         raise ValueError("time must be a nonempty finite one-dimensional
array")
80     if np.any(np.diff(time) <= 0):
81         raise ValueError("time must be strictly increasing")
82     angles = _finite_array(self.joint_angles, (len(time), 3), "
joint_angles")
83     rates = _finite_array(self.joint_velocities, (len(time), 3), "
joint_velocities")
84     return time, angles, rates
85
86 def clubhead_speed(self) -> Array:
87     """Clubhead speed at each sample, from the tip Jacobian.
88
89     The tip velocity is 'sum_i qdot_i * (z_hat x r_i)', where 'r_i'
runs
90     from joint 'i' to the tip. Its magnitude depends on the
configuration
91     through the angles between those contributions -- which is precisely
what
92     a scalar sum of 'qdot_i * |r_i|' throws away.
93     """
94     _, angles, rates = self._require_kinematics()
95     model = self.model()
96     evaluate = np.vectorize(model.clubhead_speed, signature="(n),(n)->()")
97     return cast(Array, evaluate(angles, rates))
98
99 def joint_acceleration_components(self) -> tuple[Array, Array]:
100     """Return signed same-state drift/control arrays, each shape (N, 3).
    In relative-angle coordinates (rad/s^2), these are -M^-1(Cv+g) and

```

```

101         M^-1 tau. Their sum is the model acceleration at each supplied state.
102         This algebra alone does not establish that input samples form a
103         solution.
104         """
105         _, angles, rates = self._require_kinematics()
106         if self.joint_torques is None:
107             raise ValueError("set joint_torques before decomposing")
108         torques = _finite_array(self.joint_torques, angles.shape, "
109         joint_torques")
110         model = self.model()
111
112         def evaluate(angle: Array, rate: Array, torque: Array) -> tuple[Array
113         , Array]:
114             """Solve both signed acceleration components at one state."""
115             mass = model.rigid_mass_matrix(angle)
116             bias = model.coriolis(angle, rate) @ rate + model.gravity_torque(
117             angle)
118             return np.linalg.solve(mass, -bias), np.linalg.solve(mass, torque
119             )
120
121         batch = np.vectorize(evaluate, signature="(n),(n),(n)->(n),(n)")
122         return cast(tuple[Array, Array], batch(angles, rates, torques))
123
124     def ztcf_decomposition(self) -> tuple[Array, Array]:
125         """Return same-state joint-acceleration norms in rad/s^2.
126
127         The historical name is retained, but no trajectory is integrated.
128         These
129         Euclidean norms do not add to net acceleration norm, do not measure
130         tip
131         acceleration, and do not partition clubhead speed or causal
132         importance.
133         """
134         drift, control = self.joint_acceleration_components()
135         return np.linalg.norm(drift, axis=1), np.linalg.norm(control, axis=1)
136
137     def summary(self) -> str:
138         """Human-readable summary of the analysis."""
139         time, _, _ = self._require_kinematics()
140         speeds = self.clubhead_speed()
141         peak = float(np.max(speeds))
142         peak_index = int(np.argmax(speeds))
143
144         lines = [
145             "=== Golf Swing Analysis Summary ===",
146             f"Duration: {time[-1] - time[0]:.3f} s",
147             f"Peak clubhead speed: {peak:.1f} m/s ({peak *
148             METRES_PER_SECOND_TO_MPH:.1f} mph)",
149             f"Peak speed time: {time[peak_index]:.3f} s",
150         ]
151
152         if self.joint_torques is not None:
153             drift, control = self.ztcf_decomposition()
154             lines.append(
155                 "Joint acceleration norm medians (rad/s^2): "
156                 f"drift {np.median(drift):.1f}, control {np.median(control)
157                 :.1f}"
158             )
159         return "\n".join(lines)

```

```

151
152 def _prescribed_motion(time: Array, duration: float) -> tuple[Array, Array,
153     Array]:
154     """Evaluate the analytic angles and their first two time derivatives."""
155     fraction = time / duration
156     angles = np.column_stack(
157         [
158             -np.pi / 2 + np.pi * fraction**2,
159             np.pi / 3 * np.sin(np.pi * fraction),
160             -np.pi / 2 * np.cos(np.pi * fraction / 2),
161         ]
162     )
163     rates = (
164         np.column_stack(
165             [
166                 2 * np.pi * fraction,
167                 np.pi**2 / 3 * np.cos(np.pi * fraction),
168                 np.pi**2 / 4 * np.sin(np.pi * fraction / 2),
169             ]
170         )
171         / duration
172     )
173     accelerations = (
174         np.column_stack(
175             [
176                 np.full_like(fraction, 2 * np.pi),
177                 -np.pi**3 / 3 * np.sin(np.pi * fraction),
178                 np.pi**3 / 8 * np.cos(np.pi * fraction / 2),
179             ]
180         )
181         / duration**2
182     )
183     return angles, rates, accelerations
184
185 def synthetic_swing(duration: float = 0.30, dt: float = 0.001) ->
186     SwingAnalysis:
187     """Return prescribed motion with matching model inverse-dynamics torques.
188
189     Positive finite duration and dt are in seconds; dt is the maximum sample
190     spacing, with both endpoints included. Analytic derivatives avoid a
191     mismatch
192     between grid spacing and numerical differentiation. Torques satisfy the
193     model
194     equations at these samples (up to model/numerical error); this is neither
195     a
196     forward-integration accuracy test nor measured or optimized human motion.
197     """
198     if not np.isfinite(duration) or duration <= 0:
199         raise ValueError("duration must be finite and positive")
200     if not np.isfinite(dt) or dt <= 0:
201         raise ValueError("dt must be finite and positive")
202     samples = int(np.ceil(duration / dt)) + 1
203     time = np.linspace(0.0, duration, samples)
204     angles, rates, accelerations = _prescribed_motion(time, duration)
205     analysis = SwingAnalysis(time=time, joint_angles=angles, joint_velocities
206                             =rates)
207     model = analysis.model()
208
209     def inverse(angle: Array, rate: Array, acceleration: Array) -> Array:

```

```

205     """Return generalized torque required for one prescribed state."""
206     return (
207         model.rigid_mass_matrix(angle) @ acceleration
208         + model.coriolis(angle, rate) @ rate
209         + model.gravity_torque(angle)
210     )
211
212     batch = np.vectorize(inverse, signature="(n),(n),(n)->(n)")
213     analysis.joint_torques = batch(angles, rates, accelerations)
214     return analysis

```

Listing 10.1: Prescribed-Motion Analysis, generated from `src/affine_control/swing_analysis.py`.

The listing is generated from the canonical module and checked for freshness. Run it with:

```

1 from src.affine_control.swing_analysis import synthetic_swing
2
3 print(synthetic_swing().summary())

```

Listing 10.2: Running the Prescribed-Motion Analysis

The summary reports duration, peak tip speed and its timestamp, plus separate medians of joint-acceleration norms in rad/s^2 . The peak is among sampled values, not a certified continuous-time maximum or a ball-impact value. The medians are descriptive sample statistics and use every supplied sample; nonuniform sampling would weight densely sampled periods more heavily. The historical method name `ztcf_decomposition` returns those norms; `joint_acceleration_components` exposes the signed vectors. Neither method integrates a counterfactual.

10.6 Connecting Mechanics, Control and Measurement

This example implements model construction, prescribed-motion inverse dynamics, tip kinematics and same-state joint decomposition. Optimization, feedback, robustness certificates, muscle estimation and variability analysis are *extensions*, each with a distinct evidence requirement.

For experimental inverse dynamics, start with synchronized measurements and a model that specifies coordinates, inertial parameters and external wrenches. Choose sampling and filtering from the signal bandwidth, noise and derivative accuracy needed; there is no universal adequate camera rate. In a whole-body model, unmeasured contact forces cannot simply be omitted without changing the inference. Report assumed forces, constraints and residuals, especially in unactuated base coordinates. Motion alone does not generally identify all external forces and internal torques simultaneously.

Net joint torque also does not uniquely specify muscle force. Moment arms, activation and contraction dynamics, passive tissues, force bounds and the chosen recruitment criterion affect that estimate. EMG can constrain an activation model; it is not a direct force or torque measurement. Refer to the biological actuation and inverse-dynamics treatments in Volume III before interpreting recruitment. A later torque response can also depend on earlier control through the current velocity, geometry and stored elastic energy.

For the ideal conservative rigid model, total mechanical energy satisfies $\dot{E} = \dot{q}^T \tau$. Internal inertial coupling redistributes motion and energy; it is not an additional energy supply.

A joint can have a large acceleration contribution while doing little instantaneous work if its rate is small. Peak ordering, acceleration attribution and joint power therefore answer different questions. Specify the system boundary before interpreting energy transferred across it.

A feedback study adds a declared policy, disturbances, sensor information and actuator constraints. A finite-time funnel must bound a set of initial states through an interval and into a defined terminal set; a calculation only at the nominal impact instant is insufficient. If contact is included, the guard and reset map must be part of the argument. A contraction condition or successful simulation should be reported only over the domain where its hypotheses or numerical checks hold.

For variability, an ensemble and a task map are essential. At a reference state, Dh separates first-order task-changing directions from its null space. Variance depends on alignment, coordinates, metric, rank and normalization by subspace dimension. Artificial torque perturbations describe the simulated ensemble; they do not establish a human neural synergy. If the task is a peak over time, its differentiability and event alignment require care. A fixed-time endpoint map is a simpler starting point.

Final Exercises

1. **Kinematic and Dynamic Consistency.** Run the example and compare the two configurations above. Verify analytic endpoint rates and the residual $M\ddot{q} + C\dot{q} + g - \tau$. Explain why the latter does not independently validate the model or its forward integrator.
2. **Two Parameter Studies.** First hold the prescribed joint motion fixed and vary club length and mass separately. Mass changes required torque but not tip kinematics; length changes both. Then construct a forward study with the same initial state and declared input policy. Compare outcomes and explain why these are different experiments.
3. **Signed Contributions.** Set $\tau = b$ at a chosen state. Check cancellation in joint space and compute the remaining $\dot{J}_p \dot{q}$ tip acceleration. Contrast vector, norm, tangential-acceleration and power measures.
4. **Release Experiment.** Integrate driven and zero-torque systems from a shared state over a declared interval. Refine the timestep, monitor energy where appropriate, and compare endpoint velocity vectors as well as speeds. Repeat for a different release time without treating either result as a universal contribution percentage.
5. **Muscle and Robustness Extensions.** Specify a muscle model, recruitment objective and feasibility checks, or a feedback policy and terminal set. Implement one bounded extension and report which conclusions depend on its assumptions. An impact study must define contact and reset behavior.
6. **Variability Extension.** Generate an ensemble under an explicit noise model, align samples and analyze a fixed-time task Jacobian. Report sample-size sensitivity and dimension-normalized variances. Explain what additional data would be needed to make a claim about human coordination.

Bibliography

- [1] Drake Contributors. Hydroelastic contact user guide. Model scope, representations and capabilities. Accessed 2026-10-05. https://drake.mit.edu/doxygen_cxx/group__hydroelastic__user__guide.html.
- [2] R. Featherstone. The calculation of robot dynamics using articulated-body inertias. *International Journal of Robotics Research*, 2(1):13–30, 1983.
- [3] R. Featherstone. *Rigid Body Dynamics Algorithms*. Springer, 2008.
- [4] D. H. Jacobson and D. Q. Mayne. *Differential Dynamic Programming*. American Elsevier Publishing Company, 1970.
- [5] O. Khatib. A unified approach for motion and force control of robot manipulators: The operational space formulation. *IEEE Journal on Robotics and Automation*, 3(1):43–53, 1987.
- [6] W. Lohmiller and J.-J. E. Slotine. On contraction analysis for non-linear systems. *Automatica*, 34(6):683–696, 1998.
- [7] J. Y. S. Luh, M. W. Walker, and R. P. C. Paul. On-line computational scheme for mechanical manipulators. *Journal of Dynamic Systems, Measurement, and Control*, 102(2):69–76, 1980. Legacy duplicate key retained for backwards compatibility.
- [8] Kevin M. Lynch and Frank C. Park. Modern robotics: Forward dynamics of open chains. Northwestern University, video supplement and transcript. Accessed 2026-10-05. <https://modernrobotics.northwestern.edu/nu-gm-book-resource/8-5-forward-dynamics-of-open-chains/>.
- [9] Kevin M. Lynch and Frank C. Park. Modern robotics: Lagrangian formulation of dynamics, part 1. Northwestern University, video supplement and transcript. Accessed 2026-10-05. <https://modernrobotics.northwestern.edu/nu-gm-book-resource/chapter-8-1-lagrangian-formulation-of-dynamics-part-1-of-2/>.
- [10] Kevin M. Lynch and Frank C. Park. Modern robotics: Space jacobian. Northwestern University, video supplement and transcript. Accessed 2026-10-05. <https://modernrobotics.northwestern.edu/nu-gm-book-resource/5-1-1-space-jacobian/>.
- [11] A. Majumdar and R. Tedrake. Funnel libraries for real-time robust feedback motion planning. *The International Journal of Robotics Research*, 36(8):947–982, 2017.
- [12] I. R. Manchester and J.-J. E. Slotine. Control contraction metrics: Convex and intrinsic criteria for nonlinear feedback design. *IEEE Transactions on Automatic Control*, 62(6):3046–3053, 2017.
- [13] D. Q. Mayne. A second-order gradient method for determining optimal trajectories of non-linear discrete-time systems. *International Journal of Control*, 3(1):85–95, 1966.

- [14] MuJoCo Contributors. MuJoCo documentation: Modeling. MJCF and URDF import. Accessed 2026-10-05. <https://mujoco.readthedocs.io/en/stable/modeling.html>.
- [15] MuJoCo Contributors. Computation: MuJoCo documentation, 2026. Algorithm and contact-model documentation; revisited 2026-10-05. <https://mujoco.readthedocs.io/en/stable/computation/index.html>.
- [16] MyoSuite Contributors. MyoSuite documentation. Platform introduction. Accessed 2026-10-05. <https://myosuite.readthedocs.io/en/stable/>.
- [17] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2nd edition, 2006. Legacy duplicate key retained for backwards compatibility.
- [18] OpenSim Contributors. OpenSim 4.3: HuntCrossleyForce class reference. Force component description and properties. Accessed 2026-10-05. https://opensim-org.github.io/opensim-moco-site/docs/1.1.0/html_user/classOpenSim_1_1HuntCrossleyForce.html.
- [19] Pinocchio Contributors. Pinocchio 2.9.2: Algorithms. Dynamics and constraint algorithms. Accessed 2026-10-05. <https://gepettoweb.laas.fr/doc/stack-of-tasks/pinocchio/jnrh2023/template/algorithm.html>.
- [20] ROS 2 Contributors. Adding physical and collision properties to a URDF model. Official documentation source. Accessed 2026-10-05. https://raw.githubusercontent.com/ros2/ros2_documentation/rolling/source/Capabilities/Simulation/URDF/Adding-Physical-and-Collision-Properties-to-a-URDF-Model.rst.
- [21] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. Legacy duplicate key retained for backwards compatibility.
- [22] J.-J. E. Slotine and W. Li. *Applied Nonlinear Control*. Prentice Hall, 1991.
- [23] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.
- [24] UpstreamDrift Contributors. UpstreamDrift: Engine interfaces and implementations. Pinned source revision 50784017c607f764e9c28db2d3c66b8211211f7a. Accessed 2026-10-05. https://github.com/D-sorganization/UpstreamDrift/tree/50784017c607f764e9c28db2d3c66b8211211f7a/src/shared/python/engine_core.
- [25] urdfdom Contributors. Joint parser for URDF. Joint types, frame defaults, axes and limits. Accessed 2026-10-05. https://raw.githubusercontent.com/ros/urdfdom/master/urdf_parser/src/joint.cpp.
- [26] urdfdom Contributors. Link parser for URDF. Inertial element parsing. Accessed 2026-10-05. https://raw.githubusercontent.com/ros/urdfdom/master/urdf_parser/src/link.cpp.